

Screenshot of serial monitor:

```
hi: 0.421404  
sup: 0.578596
```

```
hi: 0.280458  
sup: 0.719542
```

```
hi: 0.672662  
sup: 0.327338
```

```
hi: 0.723832  
sup: 0.276168
```

```
hi: 0.570330  
sup: 0.429670
```

```
hi: 0.779183  
sup: 0.220817
```

Python editing:

In the notebook, I changed the compiler line so that the loss was categorical crossentropy, and the metric was accuracy.

```
#model.compile(optimizer='rmsprop', loss='mse', metrics=['mae'])  
model.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['accuracy'])
```

The two changes made were replacing the loss function with categorical cross-entropy and changing the evaluation metric to accuracy. Previously, the model used mean squared error (MSE) as the loss function and mean absolute error (MAE) as the evaluation metric. While these choices are effective for regression problems, they are not the best choice for classification problems. In this model, the inputs were one-hot encoded, and the objective was binary classification, meaning the model had to choose between two possible classes. Since MAE is designed to measure prediction error for continuous values, it is not an appropriate metric for evaluating classification performance. Accuracy is a more suitable metric because it directly measures the percentage of correctly classified samples. Similarly, categorical cross-entropy is specifically designed for classification problems involving one-hot encoded labels. It measures how well the predicted probability distribution matches the true class distribution, making it more effective for optimizing classification models. Although MSE can still be used for classification, it is better suited for regression tasks and typically results in slower learning and lower classification performance compared to categorical cross-entropy.

The last fix was changing the graph label to accuracy.